

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include "../include/estruturas.h"
#include "../include/dsl.h"
#include "../include/logica.h"
#include "../include/interface.h"
#include "../include/reutilizado.h"

//
// Montagem do tabuleiro de jogo
//

// Função que preenche uma pilha do jogo com as cartas
void preencher_pilha(Pilha *p, Carta *deck, int *c_idx) {
    int c = 0;
    while (c < p->quantidade) {
        p->cartas[c] = deck[*c_idx];
        (*c_idx)++;
        c++;
    }
}

// Função que percorre as pilhas e chama a função anterior para as
preencher com cartas
void distribuir_cartas_deck(EstadoJogo *e, Carta *deck) {
    int c_idx = 0, p = 0;
    while (p < e->total_pilhas) {
        preencher_pilha(&e->pilhas[p], deck, &c_idx);
        p++;
    }
}

// Função que agrega todas as outras funções para a preparação do jogo
int preparar_jogo(EstadoJogo *e, char *caminho) {
    printf("1. A iniciar carregar_paciencia...\n");
    if (!carregar_paciencia(caminho, e)) {
        printf("Falhou no carregar_paciencia!\n");
        return 0;
    }
    printf("2. Ficheiro carregado com sucesso!\n");
    Carta *deck = criarBaralho(e->num_baralhos);
    baralharCartas(deck, e->num_baralhos * 52);
    printf("3. A distribuir cartas...\n");
    distribuir_cartas_deck(e, deck);
    printf("4. Tudo pronto!\n");
    free(deck);
    return 1;
}

//
// Validação e movimentos
//

```

Avança $\left[\begin{array}{l} (*c_idx)++; \\ c++; \end{array} \right.$ $\leftarrow$ Cartas e

$\rightarrow$ igual a carta que está no deck com o índice

$\rightarrow$ A pontuação que sabe onde ficou para não dar a mesma carta duas vezes

$\left. \begin{array}{l} \text{Se não conseguir carregar da} \\ \text{erro} \end{array} \right\}$

$\rightarrow$ Distribui as cartas

$\leftarrow$ cria os baralhos

$\leftarrow$ baralho as cartas

$\leftarrow$ como as cartas já estão nas pilhas pode libertar o deck


```
// Função que cria a copia do estado de jogo e depois executa o movimento
das cartas
void efetuar_movimento(EstadoJogo **jogo, int id_o, int id_d, int q, char
*msg) {
    *jogo = guardar_estado(*jogo);
    executar_movimento_real(&(*jogo)->pilhas[id_o], &(*jogo)->pilhas[id_d],
    q);
    sprintf(msg, "\033[32m[OK] Movimento realizado com sucesso!\033[0m");
}
```

```
// Função que processa a jogada e chama as funções necessárias
void processar_jogada(EstadoJogo **jogo, char *msg) {
    int o, d, q;
    pedirJogada(&o, &d, &q);
    int id_o = o - 1, id_d = d - 1;
    if (id_o < 0 || id_o >= (*jogo)->total_pilhas || id_d < 0 || id_d >=
    (*jogo)->total_pilhas) {
        sprintf(msg, "\033[31m[ERRO] Pilha inexistente!\033[0m");
        return;
    }
    char *flags = encontrar_regra(*jogo, &(*jogo)->pilhas[id_o],
    &(*jogo)->pilhas[id_d]);
    if (verificar_movimento(&(*jogo)->pilhas[id_o], &(*jogo)->pilhas[id_d],
    flags, q)) {
        efetuar_movimento(jogo, id_o, id_d, q, msg);
    } else {
        sprintf(msg, "\033[31m[ERRO] Movimento inválido pelas
        regras!\033[0m");
    }
}
```

```
// Função que testa as regras e move as cartas para 2 pilhas
int tentar_auto_movimento(EstadoJogo **j, int o, int d, char *msg) {
    int r = 0;
    while (r < (*j)->total_auto_regras) {
        if (strcmp((*j)->auto_regras[r].origem, (*j)->pilhas[o].nome_tipo)
        == 0 &&
        strcmp((*j)->auto_regras[r].destino, (*j)->pilhas[d].nome_tipo)
        == 0) {
            char *flags = (*j)->auto_regras[r].flags;
            if (strchr(flags, '+') != NULL) {
                if ((*j)->pilhas[o].quantidade >= 13 &&
                verificar_movimento(&(*j)->pilhas[o], &(*j)->pilhas[d],
                flags, 13)) {
                    efetuar_movimento(j, o, d, 13, msg);
                    return 1;
                }
            }
            else {
                if (verificar_movimento(&(*j)->pilhas[o], &(*j)->pilhas[d],
                flags, 1)) {
                    efetuar_movimento(j, o, d, 1, msg);
                    return 1;
                }
            }
        }
        r++;
    }
}
```



```

    r++;
}
return 0;
}

// Função que executa o movimento automático
void executar_movimentos_automaticos(EstadoJogo **jogo) {
    int o, d, moveu = 1;
    char msg_lixo[256];
    while (moveu == 1) {
        moveu = 0;
        o = 0;
        while (o < (*jogo)->total_pilhas && moveu == 0) {
            if ((*jogo)->pilhas[o].quantidade > 0) {
                d = 0;
                while (d < (*jogo)->total_pilhas && moveu == 0) {
                    if (o != d && tentar_auto_movimento(jogo, o, d,
                        msg_lixo)) {
                        moveu = 1;
                    }
                    d++;
                }
            }
            o++;
        }
    }
}

// Interface e comandos
//

// Processa a jogada e apaga qualquer dica que esteja no ecrã
void tratar_movimento(EstadoJogo **j, char *msg, int *do_, int *dd_) {
    processar_jogada(j, msg);
    executar_movimentos_automaticos(j);
    *do_ = -1;
    *dd_ = -1;
}

// Função que realiza o UNDO
void tratar_undo(EstadoJogo **j, char *msg, int *do_, int *dd_) {
    *j = fazer_undo(*j);
    sprintf(msg, "\033[33m[INFO] Undo realizado.\033[0m");
    *do_ = -1;
    *dd_ = -1;
}

// Função que guarda o progresso do jogo em save.txt
void tratar_save(EstadoJogo *j, char *cam, char *msg) {
    if (guardar_jogo(j, cam, "save.txt")) {
        sprintf(msg, "\033[32m[OK] Jogo guardado.\033[0m");
    } else {
        sprintf(msg, "\033[31m[ERRO] Falha ao guardar.\033[0m");
    }
}

```

Handwritten notes:

- Faz um ciclo por todas as pilhas para tentar fazer os movimentos automaticos* (pointing to the while loop in `executar_movimentos_automaticos`)
- Enquanto não mover* (pointing to `moveu == 0`)
- Pilhas não vazias* (pointing to `quantidade > 0`)
- tenta mover* (pointing to `tentar_auto_movimento`)
- Se tiver movido já para* (pointing to `d++`)
- Processa a jogada* (pointing to `processar_jogada`)
- tenta movimentos auto.* (pointing to `executar_movimentos_automaticos`)
- dicas* (pointing to `*do_ = -1; *dd_ = -1;`)
- Faz Undo* (pointing to `fazer_undo`)
- guarda o jogo* (pointing to `guardar_jogo`)


```
}
```

```
// Função que pede ao motor de jogo uma dica e imprime se há ou não movimentos disponíveis
```

```
void executar_comando_dica(EstadoJogo *jogo, int *d_o, int *d_d, char *msg)
{
    if (pedir_dica(jogo, d_o, d_d)) {
        sprintf(msg, "\033[33m[DICA] Mover da C%d para a C%d!\033[0m", *d_o
            + 1, *d_d + 1); } caso haja dicas
    } else {
        sprintf(msg, "\033[31m[AVISO] Sem movimentos possíveis!\033[0m");
        *d_o = -1; *d_d = -1; } Se não houver dicas é porque não há movimentos possíveis
    }
}
```

```
// Função que reencaminha para a correta de acordo com o input do jogador
```

```
void executar_comando_jogador(int cmd, EstadoJogo **j, char *msg, int *do_,
    int *dd_, int *loop, char *cam) {
    if (cmd == 0) *loop = 0;
    if (cmd == 1) tratar_movimento(j, msg, do_, dd_);
    if (cmd == 2) tratar_undo(j, msg, do_, dd_);
    if (cmd == 3) tratar_save(*j, cam, msg);
    if (cmd == 4) executar_comando_dica(*j, do_, dd_, msg);
}
```

faz o reencaminhamento

```
// Função que a cada jogada verifica se há uma vitória
```

```
void verificar_estado_vitoria(EstadoJogo *jogo, int *a_jogar) {
    if (*a_jogar && verificar_vitoria(jogo)) { Se houve vitória
        system("clear"); limpa ecrã
        vitoria();
        *a_jogar = 0;
    }
}
```

```
// Função que imprime o tabuleiro apos cada jogada
```

```
void impressao(EstadoJogo **j, int *loop, char *msg, int *do_, int *dd_,
    char *cam) {
    int cmd;
    system("clear");
    if (msg[0] != '\0') { Se tem mensagem imprimir
        printf("%s\n", msg);
        msg[0] = '\0'; Retira a mensagem da memória
    }
    mostrarTabuleiro(*j, *do_, *dd_); mostra o tabuleiro
    printf("\n[1] Mover | [2] Undo | [3] Save | [4] Dica | [0] Sair\nEscolha: ");
    if (scanf("%d", &cmd) == 1) { Se ler corretamente
        executar_comando_jogador(cmd, j, msg, do_, dd_, loop, cam); } Comando do jogador
    } else {
        int c;
        while ((c = getchar()) != '\n' && c != EOF); Enquanto não chegar ao fim dos caracteres inseridos remove um a um
        sprintf(msg, "\033[31m[ERRO] Comando inválido! Insere um número.\033[0m");
    }
    verificar_estado_vitoria(*j, loop); Verifica vitória
}
```

End of File. Ou seja quando o jogador fecha o terminal

guarda a mensagem